



OVERVIEW

This note covers the four items requested in the assignment: the **agent architecture**, **agent responsibilities**, the **RAG flow**, and **key limitations**. The system answers learner queries by routing each question to the right specialist agent and grounding the answer in an admin-curated knowledge base.

1. Agent architecture

The system uses an **orchestrator** → **specialist** pattern. Rather than one monolithic prompt, a lightweight router classifies each query and dispatches it to the specialist best suited to answer it. Each specialist retrieves only from its own slice of the knowledge base.

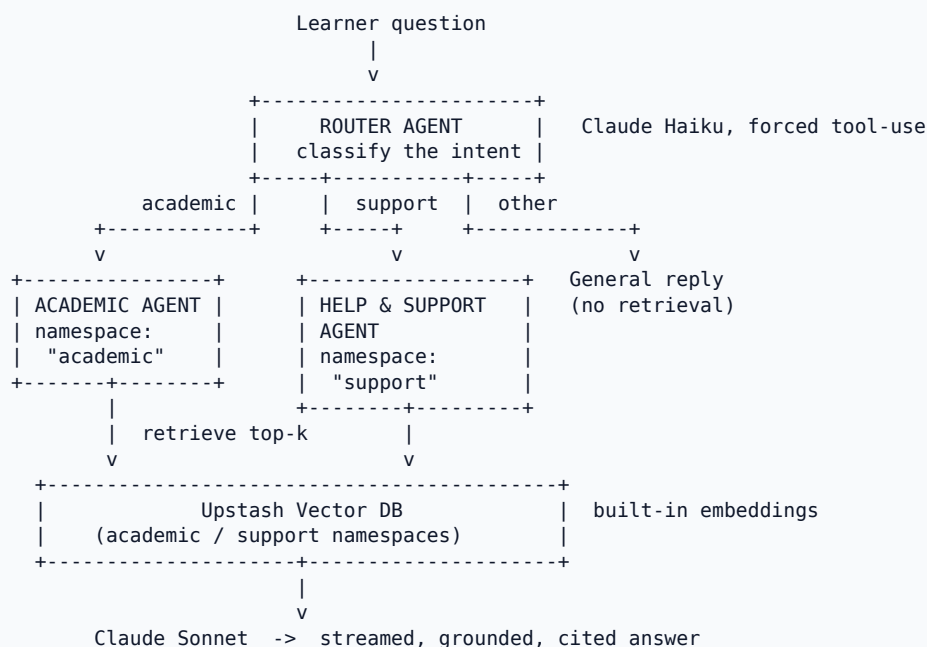


Fig 1. Orchestrator routes each query to one specialist, which retrieves from its own namespace.

Why multi-agent rather than a single prompt?

- **Separation of corpora.** Academic and support/policy content live in separate vector namespaces, so an assignment question never retrieves a refund policy and vice-versa — improving precision.
- **Specialized behaviour.** The Academic agent *coaches* (hints over hand-outs); the Support agent is *direct and procedural*. Different system prompts encode that.
- **Cost & latency.** Classification is a tiny, fast call on a cheap model (Haiku); only the final answer uses the stronger model (Sonnet).
- **Extensibility.** Adding a third agent (e.g. “Careers”) is a new namespace + persona, not a rewrite.

Implementation. Orchestration runs server-side in a single streaming endpoint (app/api/chat/route.ts). The router uses Claude with a forced route tool so the output is a strict {category, confidence, reasoning} object; a deterministic keyword fallback guarantees the chat never hard-fails if the routing call errors.

2. Agent responsibilities

Agent	Responsibility	Model	Retrieves from
Router / Orchestrator	Classify intent into academic , support , or other ; return confidence + a one-line rationale; dispatch to the right specialist.	claude-haiku-4-5	—
Academic Agent	Explain concepts simply, build intuition, and guide learners through assignments / projects / quizzes — coaching toward the answer rather than handing over graded solutions. Cites sources.	claude-sonnet-4-6	academic namespace
Help & Support Agent	Resolve certificates, schedules, live-class & platform access, payments, refunds, and account logistics; give exact steps/policy and escalate to a human when account-specific data is needed. Cites sources.	claude-sonnet-4-6	support namespace
General (fallback)	Handle greetings / off-topic messages: reply briefly and steer the learner toward what the assistant can help with. No retrieval.	claude-sonnet-4-6	—

Shared guardrails. Answer **from the retrieved context**, cite sources inline as [1], [2], and **never fabricate** specifics (dates, prices, policies). If the context lacks the answer, say so and give safe general guidance.

3. RAG flow

Ingestion (admin side)

- **Upload** a PDF / DOCX / TXT / Markdown file (or paste text) in /admin, tagged Academic or Help & Support.
- **Extract** plain text — unpdf for PDF, mammoth for DOCX, direct read for text.
- **Chunk** into ~1,100-character overlapping windows on paragraph boundaries so coherent passages embed together.
- **Embed + store** — each chunk is upserted into the category's **namespace** in Upstash Vector using its **built-in** embedding model (no separate embeddings key), with rich metadata.

Retrieval & generation (learner side)

```
Query
-> Router classifies the latest message (academic | support | other)
-> Retrieve top-k chunks from that agent's namespace
    (keep only matches above the similarity threshold => "grounded")
-> Assemble numbered [1..n] context into the specialist's system prompt
-> Claude Sonnet streams a grounded answer with inline citations
-> UI receives NDJSON events:  status -> meta -> delta... -> done
    (renders agent badge, streamed markdown, and cited sources)
```

Grounding contract: the presence of sources is equivalent to “the answer is grounded.” When no chunk clears the threshold, the model is told no documents were found and instructed to avoid inventing specifics.

4. Key limitations

This is a working **prototype**, optimised for clarity and quick deployment. Known limitations and the path to production:

- **Authentication.** The admin console uses a single shared password (HMAC-signed cookie); learners are unauthenticated. Production needs real accounts / SSO and role-based access.
- **Document management at scale.** Listing/deleting scans the index via `range()` — fine for hundreds of docs; back it with a metadata table (e.g. Postgres) for large corpora.
- **Retrieval quality.** No re-ranking, hybrid (keyword+vector) search, or query rewriting yet — all natural next steps.
- **Single-label routing.** Each query goes to exactly one agent; genuinely mixed queries route to the dominant intent. A production version could fan out and merge.
- **No conversation persistence.** History lives in the browser session only — no analytics, audit trail, or cross-device resume.
- **No evaluation harness.** Adding a golden-question set (query → expected sources) would catch regressions.
- **Parsing fidelity.** Scanned or multi-column PDFs and complex tables may extract imperfectly; OCR + layout-aware parsing would help.

What this prototype demonstrates

A complete, deployable loop: **admin ingestion → embedding → namespaced vector store → intent routing → grounded, cited, streaming answers** — with a clean separation between the Academic and Help & Support domains and a UI that makes the multi-agent behaviour visible to the learner.